Übungsblatt 5: Der Single-Cycle-Datenpfad

Mikrocomputertechnik 1

Musterlösung

Zielsetzung

In diesem Übungsblatt öffnen wir die Blackbox des Prozessors. Sie lernen den Ripes-Simulator kennen und analysieren, wie eine Zeile Assembler physikalisch den Stromfluss auf dem Silizium-Chip steuert. Sie identifizieren Multiplexer, lesen Steuersignale ab und verfolgen den Weg von Daten durch die ALU und den Arbeitsspeicher.

Aufgabe 1: Ripes Onboarding und R-Type-Trace

Starten Sie den Ripes-Simulator (Web-Version oder lokal) und stellen Sie oben links zwingend den **Single-Cycle Processor** ein.

Geben Sie den folgenden Code in den Editor-Tab ein:

```
.text
li t0, 50
li t1, 100
add a0, t0, t1
```

Klicken Sie sich im Processor-Tab mit der Clock-Taste vorwärts, bis der `add`-Befehl im Datenpfad anliegt (der PC zeigt auf den `add`-Befehl). Analysieren Sie nun die Hardware-Leitungen.

Register File: RD1 und RD2

Welche Hexadezimal-Werte liegen an den Lese-Ports RD1 und RD2 des Register Files an? (Nutzen Sie ggf. Tooltips beim Darüberfahren mit der Maus.)

Steuersignal `ALUSrc`

Welchen Wert (0 oder 1) hat das Steuersignal für den MUX vor dem B-Eingang der ALU (`ALUSrc`)? Warum muss der MUX genau so stehen?

Steuersignal `RegWrite`

Welchen Wert hat das Steuersignal `RegWrite` am Register File? Was würde physikalisch passieren, wenn dieses Signal auf 0 stehen würde?

Ihre Lösung

(Notieren Sie Ihre abgelesenen Signale hier.)

Musterlösung Aufgabe 1

RD1 und RD2:

- An RD1 liegt `0x32` (Dezimal 50 aus `t0`).
- An RD2 liegt `0x64` (Dezimal 100 aus `t1`).

`ALUSrc`:

- Das Signal `ALUSrc` steht auf 0.
- Der MUX muss den Wert aus dem Register File (RD2) zur ALU durchlassen, da `add` eine reine Register-Rechnung (R-Type) ist und keine Konstante aus dem ImmGen benötigt.

`RegWrite`:

- Das Signal `RegWrite` steht auf 1.
- Bei 0 würden die berechneten Daten der ALU (`0x96` bzw. 150) am Write-Data-Port des Register Files abprallen; `a0` würde beim Taktende nicht überschrieben.

Aufgabe 2: Der kritische Pfad (Load-Trace)

Löschen Sie den alten Code und testen Sie einen Load-Befehl:

```
.data
werte: .word 99, 88

.text
la s1, werte
lw a0, 4(s1)
```

Klicken Sie die Clock, bis der `lw`-Befehl aktiv ist.

ImmGen

Betrachten Sie den Baustein ImmGen (Immediate Generator). Aus dem 32-Bit-Befehlswort fließen Bits in dieses Bauteil. Welcher 32-Bit-Hex-Wert verlässt den ImmGen?

ALU als Adressrechner

Welche Aufgabe hat die Haupt-ALU bei diesem `lw`-Befehl? Welche Hex-Adresse berechnet sie an ihrem Ausgang?

Multiplexer `MemtoReg`

Betrachten Sie den Write-Back-Multiplexer ganz rechts vor dem Register File (P&H: `MemtoReg`; in Ripes oft ein 3-Wege-MUX). Auf welchen Daten-Pfad hat das Steuerwerk geschaltet und warum (ALU vs. Speicher)?

Ihre Lösung

(Notieren Sie Ihre abgelesenen Signale hier.)

Musterlösung Aufgabe 2

ImmGen:

- Der ImmGen extrahiert das 12-Bit-Offset aus dem Befehlswort und führt eine Vorzeichenerweiterung auf 32 Bit durch.
- Bei `lw a0, 4(s1)` ist das Offset +4; der ImmGen liefert `0x00000004`.

ALU als Adressrechner:

- Die ALU berechnet die Ziel-RAM-Adresse (Base + Offset), keine Nutzer-Arithmetik.
- Sie addiert die Basisadresse aus `s1` (in Ripes typischerweise `0x10000000`) mit der 4 aus dem ImmGen.
- Der ALU-Ausgang zeigt damit z. B. `0x10000004` (exakte Basisadresse ggf. im Memory-Tab prüfen).

`MemtoReg`:

- Der MUX `MemtoReg` steht auf 1.
- Er blockiert das Adress-Ergebnis der ALU und schaltet den Daten-Pfad aus dem RAM (Read Data) zum Register File durch, damit der Wert 88 in `a0` landet.

Aufgabe 3: Der Branch-Trace (Fall-Through vs. Taken)

Löschen Sie den Code und analysieren Sie das Sprungverhalten (Branches):

```
.text
li t0, 5
li t1, 10
beq t0, t1, Label
add a0, t0, t1
Label:
sub a0, t1, t0
```

Fall-Through

Steppen Sie bis zum `beq`-Befehl. Die Register `t0` und `t1` sind ungleich.

1. Wird der Branch genommen oder nicht? (Beobachten Sie den PC und die Branch-Vergleichseinheit — Ripes hat oft **keinen** klassischen ALU-Zero+AND wie im P&H-Diagramm.)
2. Welche Adresse (in Hex) wird bei der nächsten Taktflanke in den PC geladen?

Branch Taken

Ändern Sie `li t1, 10` zu `li t1, 5`, sodass die Bedingung wahr wird. Steppen Sie wieder zum `beq`.

1. Was ändert sich am PC-Verhalten?
2. Welches Bauteil liefert die Sprungzieladresse in **Ripes** (Hinweis: oft die Haupt-ALU mit PC als Operand — nicht zwingend ein separater Branch-Addierer wie im Lehrbuchdiagramm)?

Ihre Lösung

(Notieren Sie Ihre Beobachtungen hier.)

Musterlösung Aufgabe 3

Fall-Through ($5 \neq 10$):

- Branch wird **nicht** genommen; der PC macht linear weiter.
- `li` assembliert typisch zu je einem `addi`; `beq` liegt bei Adresse `0x08` $\rightarrow$ nächster PC = PC$+4$ = `0x0000000C` (Ripes-Textsegment startet bei `0x0`).

Branch Taken ($5 = 5$):

- Branch wird genommen; PC springt auf das Label `Label`.
- In Ripes berechnet typischerweise die **Haupt-ALU** (mit PC als Operand) das Branch-Target — nicht der separate Branch-Addierer aus dem Patterson/Hennessy-Diagramm. Mapping siehe Vorlesung 5.